

INTERVIEW PREPARATION SERIES

LeetCode Practice Set

Control Flow: Conditionals & Loops

10 Curated Problems Across 4 Tiers • Language-Independent •
Conditional Statements, Loops, Nested Loops & Break/Continue

HOW TO USE THIS DOCUMENT

Every **problem title**, **LEETCODE NNN** badge, and number in the summary table is **clickable** — opens directly on leetcode.com. Problems are language-independent: solve them in Java, Python, or C++. Ordered by control flow complexity, not difficulty.

TIER 01

Conditional Logic — Single Loop & Branching

Problems where the condition inside the loop is the entire challenge. Get the branch right; the rest is trivial.

PROBLEM 01 • [LEETCODE 1](#)

Two Sum ↗

CONTROL FLOW FOCUS • if-else inside loop • $O(n^2) \rightarrow O(n)$ via conditional elimination • Correctness of nested branch

The Problem: Find two indices in an array that sum to a target. Return the indices.

Why It's Here: The nested `if` to detect a pair is trivial. The interesting control flow is **replacing the inner loop entirely** with an $O(1)$ conditional: “is complement already seen?”. The interview tests whether you can recognise that a conditional replaces a loop.

What They Expect at 3 YOE: State the brute force $O(n^2)$ first. Two nested loops, one conditional: `if arr[i] + arr[j] == target`. Then collapse the inner

loop: build a hash map, and for each element x check `if (target - x) in map`. The conditional remains; the inner loop disappears.

Control Flow Insight: A lookup-based conditional replaces iteration. The pattern “for each x , check if something is already seen” is the canonical $O(n^2) \rightarrow O(n)$ transformation. Recognising the conditional as the bottle neck, not the loop count, is the skill.

Follow-up: “What if you cannot use a hash map?” — Sort + two pointers. Two conditionals (`if sum < target: left++` else `right--`) replace the nested loop.

PROBLEM 02 • [LEETCODE 268](#)

Missing Number ↗

CONTROL FLOW FOCUS • XOR loop • Conditional-free $O(n)$ • Sum formula replaces loop

The Problem: Given n distinct numbers in range $[0, n]$, find the missing one.

Why It's Here: Three solutions, three different control flow styles: (1) sort + loop with `if arr[i] != i` conditional; (2) sum formula — no conditional, $O(1)$; (3) XOR — no conditional, bit manipulation. The progression shows how recognising mathematical structure eliminates branching entirely.

What They Expect: Sum formula: expected sum = $n(n+1)/2$. Actual sum = $\sum arr[i]$. Missing = expected – actual. $O(n)$ time, $O(1)$ space, zero conditionals. XOR alternative: XOR all indices $0..n$ with all array values. All pairs cancel; the missing number remains.

Control Flow Insight: The best algorithm here has no conditional at all. When a mathematical identity eliminates branching, always prefer it: fewer paths = fewer bugs, fewer edge cases, and easier testing. The ‘boring’ $O(n)$ solution without branches is production code.

PROBLEM 03 • [LEETCODE 283](#)

Move Zeroes ↗

CONTROL FLOW FOCUS • Two-pointer with in-loop conditional • $O(n)$ • In-place condition

The Problem: Move all zeros to the end of an array in-place, preserving non-zero order.

Why It's Here: The conditional inside the loop — `if arr[i] != 0` — drives the pointer logic. Tests whether you can write a loop where the condition controls *state* (the slow pointer) rather than just branching to different code paths.

What They Expect: Two-pointer approach. Slow pointer tracks the next write position. Fast pointer scans. Condition: `if arr[fast] != 0`, write to slow and advance slow. After full scan, fill remaining with zeros. $O(n)$ time, $O(1)$ space.

Control Flow Insight: The conditional inside a loop can advance auxiliary state, not just branch execution. The slow pointer is updated only inside the true branch. This 'conditional pointer advance' pattern is the basis of partition (quicksort), Dutch National Flag, and all two-pointer problems.

Follow-up: "Minimise total writes." — Swap instead of copy: `swap(arr[fast], arr[slow])` avoids the zero-fill pass. Total writes = $2k$ where k = non-zero elements.

TIER 02

Loop Termination & State Machines

Problems where the termination condition or the loop state is the tricky part, not the body logic.

PROBLEM 04 • LEETCODE 26

Remove Duplicates from Sorted Array ↗

CONTROL FLOW FOCUS • In-place single loop • Conditional pointer advance • $O(n)$ invariant maintenance

The Problem: Remove duplicates in-place from a sorted array, return the new length. Elements beyond the new length don't matter.

Why It's Here: Classic loop-invariant problem. You must maintain an invariant (all elements at indices $0..k-1$ are unique) through every loop iteration. The conditional `if arr[i] != arr[k-1]` determines whether to advance the write pointer. This is the in-place conditional pattern at its clearest.

What They Expect: Single pass, two pointers. Write pointer k starts at 1. Read pointer i from 1 to $n-1$. Condition: `if arr[i] != arr[k-1]`: write `arr[i]` at `arr[k]`, advance k . $O(n)$ time, $O(1)$ space. Loop invariant: `arr[0..k-1]` contains unique elements in sorted order at all times.

Control Flow Insight: Loop invariants make conditional logic provably correct. Before stating complexity, state the invariant. “At the end of every iteration, `arr[0..k-1]` contains the unique prefix seen so far.” An interviewer who pushes back on correctness is asking you to prove the invariant holds through the conditional.

PROBLEM 05 • LEETCODE 485

Max Consecutive Ones ↗

CONTROL FLOW FOCUS • Single loop, reset pattern • if-else drives counter state • $O(n)$

The Problem: Given a binary array, find the maximum number of consecutive 1s.

Why It's Here: Deceptively simple, but the control flow has a subtle reset: the counter must reset to 0 on a 0, and the max must be updated on every 1 (or after each run). Candidates who update max only at 0 miss the case where the longest run is at the end of the array.

What They Expect: Two variables, one loop, one if-else. `current = 0, max_run = 0`. For each element: `if elem == 1: current += 1; max_run = max(max_run, current)`. Else: `current = 0`. The max update *inside* the if branch handles the end-of-array case.

Control Flow Insight: Where you update the max inside the conditional matters for edge cases. Updating inside the `if` instead of after the `else` is the difference between correct and off-by-one at the array boundary. The interviewer will test with arrays ending in 1s to catch this.

PROBLEM 06 • LEETCODE 31

Next Permutation ↗

CONTROL FLOW FOCUS • Three-phase loop logic • Right-to-left traversal • Conditional swap + reverse

The Problem: Rearrange numbers into the lexicographically next permutation in-place. If the largest, wrap around to the smallest.

Why It's Here: Three phases, each a loop with a specific conditional termination. (1) Find the rightmost descent: `while i >= 0 and arr[i] >= arr[i+1]`. (2) Find the rightmost element larger than `arr[i]`: `while arr[j] <=`

`arr[i]`. (3) Swap and reverse. The problem tests reading algorithm logic that is entirely defined by loop conditions.

What They Expect: Phase 1: scan right-to-left to find the first pair where `arr[i] < arr[i+1]`. If no such pair, array is in descending order — reverse entirely. Phase 2: scan right-to-left from end to find the smallest element `> arr[i]`. Swap. Phase 3: reverse `arr[i+1..end]` to get the smallest suffix.

Control Flow Insight: Multi-phase loops with independent termination conditions are the hardest to get right. Each loop's exit condition defines a structural property of the array. Interviewers probe edge cases: `[3,2,1]` (already descending), `[1,2,3]` (ascending, next = `[1,3,2]`), and single-element arrays.

TIER 03

Nested Loop Patterns — Two Pointers & Matrix Traversal

Problems that look like $O(n^2)$ nested loops but have $O(n)$ solutions — the transformation is the lesson.

PROBLEM 07 • [LEETCODE 941](#)

Valid Mountain Array ↗

CONTROL FLOW FOCUS • Two-phase single loop • State machine with boolean conditions • $O(n)$

The Problem: Check if an array is a valid mountain: strictly increases then strictly decreases, with peak not at endpoints.

Why It's Here: A state machine implemented with loop conditions. The array transitions through states: ascending → peak → descending. Each state has a specific exit condition. Tests whether you can implement a multi-state conditional loop cleanly.

What They Expect: Two-pointer approach from both ends. Left pointer climbs from index 0 while `arr[left] < arr[left+1]`. Right pointer climbs from end while `arr[right] < arr[right-1]`. Valid mountain: `left == right` (same peak), `left > 0`, `right < n-1`.

Control Flow Insight: Meeting-in-the-middle loop termination as a correctness condition. The `left == right` check at the end is the validity gate

— if they meet at the same point, both climbs terminated at the peak. If left goes beyond right, or they stop at different indices, the shape is wrong. One equality check replaces a complex multi-conditional validation.

PROBLEM 08 • [LEETCODE 56](#)

Merge Intervals [↗](#)

CONTROL FLOW FOCUS • Sort-then-loop • Conditional extend vs append • $O(n \log n)$

The Problem: Merge overlapping intervals in a list.

Why It's Here: A single loop after sorting, but the conditional inside determines whether to *extend* the last interval or *start* a new one. Tests whether you handle the conditional correctly at each iteration — specifically the boundary condition (`end >= interval.start`) that determines overlap.

What They Expect: Sort by start, single pass. If the current interval overlaps the last merged (`current.start <= last.end`), extend: `last.end = max(last.end, current.end)`. Else, append. The `max()` in the extension handles the case where one interval fully contains another.

Control Flow Insight: The conditional inside the loop must handle all three cases: no overlap, partial overlap, full containment. The `max(last.end, current.end)` handles full containment implicitly. Candidates who write `last.end = current.end` without `max()` fail on `[1,10]`, `[3,4]` (should stay `[1,10]`, not become `[1,4]`).

TIER 04

Nested Loops — Matrix Operations & In-Place Transforms

Problems requiring coordinated nested loops with precise boundary conditions. Every off-by-one matters.

PROBLEM 09 • [LEETCODE 73](#)

Set Matrix Zeroes ↗

CONTROL FLOW FOCUS • Two-pass nested loop • Boolean conditional encoding • $O(m \times n)$ time • $O(1)$ space

The Problem: If a cell in an $m \times n$ matrix is 0, set its entire row and column to 0. Do it in-place.

Why It's Here: The naive nested loop is correct but $O(m \times n)$ space. The $O(1)$ space solution requires using the first row and column as markers — a conditional encoding trick. The first row is *both* the marker for other rows *and* a row that might need zeroing itself. Managing this ambiguity with a single boolean and two passes of nested loops is the challenge.

What They Expect: Pass 1: check if row 0 and col 0 themselves contain zeros (save in two booleans). Use the first row and column as markers for other rows/columns. Pass 2: iterate the matrix body (rows 1..m, cols 1..n) and apply zeros. Pass 3: zero row 0 and col 0 from the booleans. Two nested loops + careful conditional ordering.

Control Flow Insight: Order of the three passes is a correctness constraint. Zeroing the first row before using it as a marker would destroy the marker information. This is a data-dependency ordering problem disguised as a matrix problem: the conditional reads (marker check) must precede the conditional writes (zero-out).

PROBLEM 10 • [LEETCODE 48](#)

Rotate Image ↗

CONTROL FLOW FOCUS • Nested loop with 4-way swap • Layer-by-layer traversal • $O(n^2)$ in-place

The Problem: Rotate an $n \times n$ matrix 90 degrees clockwise in-place.

Why It's Here: The core is a nested loop where the *indices* of the four cells involved in each swap are derived from the loop variables. Tests whether you can reason about index arithmetic inside a nested loop and get all four coordinates right.

What They Expect: Layer-by-layer rotation. Outer loop: layer 0 to $n//2$. Inner loop: i from layer to $n-1$ -layer. Four-position swap: top $\rightarrow$ right $\rightarrow$ bottom $\rightarrow$ left $\rightarrow$ top. Coordinates: (layer, i), (i , $n-1$ -layer), ($n-1$ -layer, $n-1-i$), ($n-1-i$, layer). Alternative: transpose + reverse each row (two nested loops instead of one with 4-way swap).

Control Flow Insight: When nested loop indices feed into position calculations, derive them from a small example first. Draw a 4×4 matrix, trace one swap manually for layer=0, $i=0$. Verify the four coordinate expressions. Then generalise. Correct index arithmetic is not memorisable — it must be derivable from the geometry. Interviewers test this derivation, not memorisation.

Follow-up: “Counter-clockwise?” — Transpose + reverse each *column* (or reverse then transpose). Same pattern, different axis.

SUMMARY

Problem Reference Table

Problem	L C	Control Flow Concept	Key Insight
Two Sum	1	if-else replaces inner loop	Conditional eliminates $O(n)$ scan
Missing Number	2 6 8	Conditional-free sum formula	Math identity removes branching
Move Zeroes	2 8 3	In-loop conditional advances pointer	Condition drives auxiliary state
Remove Duplicates	2 6	Loop invariant via conditional	State maintained through condition

Problem	L C	Control Flow Concept	Key Insight
Max Consecutive Ones	4 8 5	Reset pattern in if-else	Update inside if handles end edge
Next Permutation	3 1	Three-phase loop termination	Each loop exit is a structural fact
Valid Mountain Array	9 4 1	State machine loop conditions	Meeting-point equality as validity
Merge Intervals	5 6	Extend vs append conditional	max() handles containment implicitly
Set Matrix Zeroes	7 3	Two-pass nested loop ordering	Read-before-write dependency order
Rotate Image	4 8	4-way swap in nested loop	Derive coordinates; never memorise

PRO TIP

For every loop, state three things before you code: (1) the loop invariant, (2) the termination condition, and (3) what happens at the boundaries (empty array, single element, all-same). Interviewers grade these explicitly at product-company onsite rounds.